B.TECH. PROJECT ON

WEARABLE SMART SYSTEM FOR VISUALLY IMPAIRED PEOPLE

Submitted by
Manas Pratim Das (2022UEC2697)
Md. Haider Imam (2022UEC2708)
Sandeep Poddar (2022UEC2694)
Sourabh (2022UEC2704)

Under the Guidance of
Dr. Ritu Raj Singh

Project-II in partial fulfillment of requirement for the award of
B.Tech. in
Electronics & Communication Engineering

Department of Electronics & Communication Engineering
NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY
NEW DELHI-110078
May- 2026

CERTIFICATE

Certified that Manas Pratim Das (2022UEC2697), Md. Haider Imam (2022UEC2708), Sandeep Poddar (2022UEC2694) and Sourabh (2022UEC2704) have carried out their project work presented in this project entitled "WEARABLE SMART SYSTEM FOR VISUALLY IMPAIRED PEOPLE" for the award of Bachelor of Technology, Department of Electronics and Communication, Netaji Subhas University of Technology, New Delhi, under my/our supervision. The project embodies results of original work, and studies are carried out by the student himself/herself and the contents of the project do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Dr. Ritu Raj Singh

Date:

ACKNOWLEDGEMENT

We would like to express my gratitude and appreciation to all those who make it possible to complete this project. Special thanks to our project supervisor Dr. Ritu Raj Singh whose help, stimulating suggestions and encouragement helped us project work. We also sincerely thank our colleagues for the time spent proofreading and correcting our mistakes.

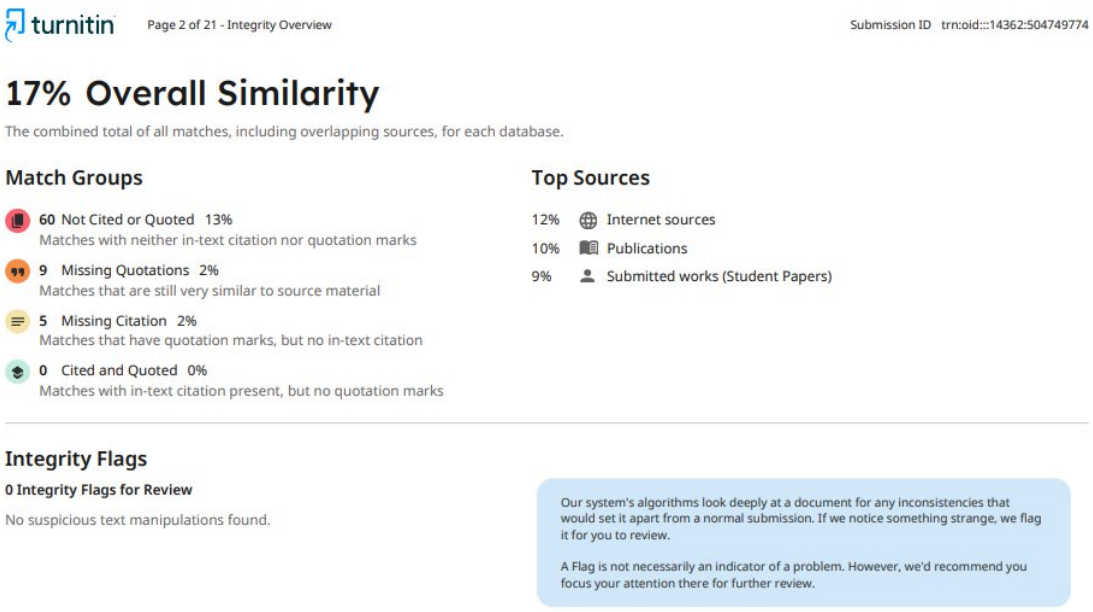
Manas Pratim Das
(2022UEC2697)

Md. Haider Imam
(2022UEC2708)

Sandeep Poddar
(2022UEC2694)

Sourabh
(2022UEC2704)

PLAGIARISM REPORT



turnitin Page 2 of 21 - Integrity Overview Submission ID trn:oid:::14362:504749774

17% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **60** Not Cited or Quoted 13%
  Matches with neither in-text citation nor quotation marks
- **9** Missing Quotations 2%
  Matches that are still very similar to source material
- **5** Missing Citation 2%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

12% Internet sources
10% Publications
9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

ABSTRACT

Visually impaired people face significant challenges in navigating and interacting with their environments. Currently, there is no robust, reliable, and affordable system that adequately addresses these issues. Such a system can be divided into distinct stages: data collection, data storage and pro-cessing, object detection, and user feedback. This paper presents a comprehensive literature review of the various technologies and methodologies employed across these workflow stages. By integrating these technologies, a practical and widely accessible assistive system can be developed for visually impaired individuals. However, several limitations remain, including the need for compact, good processing power, and low-heat-generating hardware; limited datasets and object classes focused primarily on indoor environments; the inability to run complex models directly on microcontrollers, requiring separate servers; and a lack of effective feedback mechanisms to the user.

CONTENT

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
AP	Access Point
BCE	Binary Cross-Entropy
BLE	Bluetooth Low Energy
CIoU	Complete Intersection over Union
FLOPs	Floating Point Operations
FPS	Frames Per Second
GPIO	General Purpose Input/Output
HTTP	HyperText Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
LiDAR	Light Detection and Ranging
MJPEG	Motion JPEG
NMS	Non-Maximum Suppression
NPU	Neural Processing Unit
R-CNN	Region-based Convolutional Neural Network
REST	Representational State Transfer
RPN	Region Proposal Network
RTOS	Real-Time Operating System
SLAM	Simultaneous Localization and Mapping
SPPF	Spatial Pyramid Pooling Fast
TCP	Transmission Control Protocol
TTS	Text-to-Speech
ToF	Time-of-Flight
Wi-Fi	Wireless Fidelity
YOLO	You Look Only Once

CHAPTER 1
INTRODUCTION

1.1 Background

Visual impairment is a highly impactful disability, affecting millions of people worldwide. According to the World Health Organization, nearly 2.2 billion individuals suffer from near or farsightedness, with approximately 300 million experiencing moderate to severe visual impairment[1]. This greatly complicates daily tasks and routines. While traditional assistive devices like canes provide tactile information about ground-level obstacles, they lack the ability to convey semantic information about the environment. Consequently, users cannot distinguish between objects such as chairs and people. Some advanced assistive technologies, such as Meta Glasses or Apple Vision, exist; however, they are often not accessible to the general population due to cost considerations. With recent developments in Internet of Things (IoT) devices and low-power object detection models, there is an opportunity to bridge this gap. This project proposes a smart wearable device that functions as an 'eye' for the user, offering auditory feedback to enhance environmental awareness.

1.2 Problem Statement

Current electronic travel authorization (ETA) systems often depend on ultrasonic sensors that produce basic beeping signals. However, these systems tend to underperform in crowded environments. There is a notable gap in affordable, low-power, real-time navigation solutions capable of accurately classifying objects and relaying information to users with minimal latency.

1.3 Objectives

The main objectives of this project are to develop a smart wearable system that can provide visually impaired users with real time information about their surrounding environments using an integrated camera module, implementing object detection, efficient communication protocols, and audio-based feedback system.

To select a lightweight and user-friendly camera module that can continuously capture the user's surroundings (video feed) with minimal power consumption and also comfortable on wearing this. For real-time object detection, we will utilize YOLOv12[2], a state-of-the-art, deep-learning-based model known for its low computational requirements.

To establish a WebSocket protocol for communication between the ESP32-CAM[3] and the server. This is essential for achieving fast, low-latency, and bidirectional communication, which in turn minimizes frame transmission and processing delays. Utilizing WebSockets[4] reduces the load on the server by eliminating continuous API update requests, thereby mitigating server bottlenecks and the communication delays

associated with them. For feedback system, transmitting information about detected objects to the user in form of audio messages through pre-recorded audio.

CHAPTER 2
LITERATURE REVIEW

2.1 Traditional Detection Used (Ultrasonic Sensors)

Traditional method of detecting objects, i.e. by using an ultrasonic sensor. An ultrasonic sensor is a device which uses ultrasonic sound transmitter and ultrasonic sound receiver. This ultrasonic sensor emits ultrasonic waves. The transmitter-receiver pair together provides the signal using which distance to the object can be calculated using mathematical computations. This signal is sensed by ESP-32 and distance is computed.

In this paper[5], the author implemented using three ultrasonic sensors (S_F, S_R, S_L) to detect more accurate objects. These sensors emit 40Khz ultrasonic waves and determine the time taken by these waves to return after encountering the obstacles. Using three sensors, objects can be detected in front, right and left side which improves the accuracy of the device. The front sensor (S_F,) monitors objects in the front, while the right sensor (S_R) is responsible for tracking objects on the right side, and the left sensor (S_L) keeps tabs on the left side. When the distance in front (F_D) falls below the safe distance (S_D) of 200cm, the ESP-32 triggers the audio amplifier, producing a distinctive low-frequency (75Hz) audio signal, accompanied by the vocal warning "Obstacle in Front." The ESP-32 also assesses the distances measured by S_L and S_R. If the left distance (L_D) is less than S_D, it commands the audio amplifier to articulate 'Turn right.' Conversely, if the right distance (R_D) is less than S_D, it directs the audio amplifier to announce "Turn
left".

In this paper[6], the author modified the ultrasonic sensor (HC-SR04) in the infrastructure module. The ultrasonic sensor in the infrastructure module does not have a transmitter unit. The author achieved by de-soldering the transmitter pins and shorting the exposed pins. Using this modification, the receiver can listen any ultrasonic sound with specific pattern. The Infrastructure Module has a dedicated microcontroller. The microcontroller is powered by a 3.7V LiPo battery. However, the ultrasonic sensor requires 5V input. The author used a boost converter to step up the voltage. Once a client establishes connection with the Infrastructure Module, it continuously polls the ultrasonic sensor to determine if the client module is directly below itself. Additionally, whenever the client is detected, the Infrastructure Module notifies the client that it is directly below that Infrastructure Module.

In this paper[7], the author implemented the obstacle detection system using ultrasonic sensor which measures both distance and echo when the object is detected. The sensor has four pins including VC, GND, Trigger (output pin) and echo (input pin). The sensor emits the waves like bat spreading the waves and listen for the echoes of nearby obstacles. It works on the principle of transmitting ultrasonic waves and receive the reflected waves. The author uses both ultrasonic sensor and buzzer which is connected to the microcontroller board to collect the data of these sensors which is particularly useful for visually impaired individuals in identifying and navigating around obstacles.

2.2 Evolution of Object Detection Model

Object detection is a crucial computational component of the proposed assistive system. The evolution of these algorithms has shifted from high-latency, two-stage detectors to efficient, single-stage detectors capable of operating on constrained hardware architectures. This section reviews the development of these models, examining their trade-offs within the context of visual impairment assistance.

2.2.1 Early Architecture and Two-stage Detectors

Initial research into deep learning-based navigation extensively utilized Region-based Convolutional Neural Networks (R-CNN). Notably, Faster R-CNN[8] emerged as a prominent architecture, employing a Region Proposal Network (RPN) combined with a secondary classifier. The loss function for Faster R-CNN[8] is given by:

$$L_{Faster\ R-CNN} = L_{cls}^{RPN} + L_{reg}^{RPN} + L_{cls}^{Det} + L_{reg}^{Det} \quad 1[8]$$

While models such as EATBP-ODHOA[8], leverage Faster R-CNN[8] to attain high precision in detecting small objects, they are hindered by substantial computational demands. Comparative studies indicate that such models can require significant processing times—up to 10.87 seconds in unoptimized environments—posing dangerous latency risks for blind users navigating dynamic settings.

2.2.2 YOLO Revolution (V3 to V7)

To mitigate the latency issues inherent in two-stage detectors, the YOLO (You Only Look Once) family was introduced, re-framing detection as a single regression problem.

- **YOLOv3 and v4:** These iterations incorporated multi-scale prediction capabilities but still had relatively high parameter counts.
- **YOLOv5:** A pivotal release employed in the SCODL-ODC[9] model, using a backbone for feature extraction and a neck component for multi-scale fusion, achieving an impressive balance between speed and accuracy. Comparative analyses demonstrate YOLOv5[10] attaining processing times as low as 2.92 seconds in optimized environments, making it far better suited for wearable applications than R-CNN.
- **YOLOv6 and v7:** YOLOv7[11], utilized in the ODSDP-ADLMSSO[12] model, introduced further improvements with the Extended Efficient Layer Aggregation Network (E-ELAN) backbone. This architecture allows the model to learn diverse features without destroying the gradient path, making it highly effective for real-time applications.

2.2.3 Recent developments (V8 to V11)

Following version 7, the evolution of YOLO has concentrated on architectural enhancements:

- **YOLOv8:** Introduced an anchor-free detection head, simplifying the training process and improving adaptability to various object shapes[13].

- **YOLOv9:** Focused on Programmable Gradient Information (PGI) to minimize information loss during deep network transmission[14].
- **YOLOv10 and v11:** Advanced efficiency by implementing Non-Maximum Suppression (NMS)-free training and improved feature extraction modules, further reducing inference latency on edge devices[15], [16].

2.2.4 Why YOLOv8?

For the core visual intelligence of the Visual Eyes navigation system, the architectural methodology underwent a critical evolution. While early iterations of this project considered standard object detection models (such as YOLOv12 and YOLOv10) for their high-speed bounding-box generation, empirical testing in real-world navigational scenarios revealed fundamental limitations in standard detection paradigms. Consequently, the system architecture was upgraded to utilize the YOLOv8 Instance Segmentation model (yolov8n-seg.pt).

This transition from rigid bounding boxes to pixel-perfect instance segmentation represents a paradigm shift in how the wearable device interprets spatial geometry. The selection of the YOLOv8-Seg architecture is grounded in its superiority for real-time mask generation, its seamless integration with region-based navigation logic, and its unparalleled quantitative inference speed on edge-cloud frameworks.

Table 2.1: Comparison of the various YOLO versions

Model	Size (pixel)	mAP (50-95)	Latency (ms)	Params (M)	FLOPs (B)
YOLO12n[2]	640	40.6	1.64	2.6	6.5
YOLO11n[16]	640	39.5	1.5	2.6	6.5
YOLOv10n[15]	640	38.5	1.84	2.3	6.7
YOLOv9s[14]	640	46.8	3.54	7.1	26.4
YOLOv8n[13]	640	37.3	0.99	3.2	8.7
YOLOv7tiny-SiLU[11]	640	38.7	3.49	6.2	13.8
YOLOv6n[17]	640	37.5	1.17	4.7	11.4
YOLOv5nu[10]	640	34.3	1.06	2.6	7.7

2.2.4.1 The Fundamental Flaw of Standard Object Detection

To understand the necessity of YOLOv8 Segmentation, one must first analyze the critical flaws of standard object detectors (like YOLOv5, YOLOv7, and YOLOv12) when applied to blind navigation. Standard object detection algorithms encapsulate detected hazards within rigid, rectangular bounding boxes. While computationally inexpensive, these rectangles introduce severe spatial inaccuracies.

A bounding box represents the absolute maximum width and height of an object, fundamentally ignoring the object's actual geometric contour. This creates the **"False Positive Occupancy"** problem. For example, consider a bicycle parked diagonally on a sidewalk, or a pedestrian walking with an extended umbrella. If the system draws a rectangular bounding box around the diagonal bicycle, nearly 40% to 50% of the area

inside that rectangle is actually safe, empty, walkable pavement.

In a traditional bounding-box system, the navigation logic interprets the entire rectangle as a solid wall. Consequently, the blind user receives a mandatory "STOP" command, even though sufficient physical space exists to walk past the obstacle. In dense, unstructured environments (such as Indian streets), bounding boxes overlap extensively, causing the system to falsely perceive the entire environment as blocked, leading to navigational paralysis and user frustration.

2.2.4.2 The Instance Segmentation Advantage

YOLOv8 Instance Segmentation[18] resolves this vulnerability by completely discarding the rectangular constraint. Instead of simply localizing an object, segmentation classifies every single pixel within the camera frame. The YOLOv8-Seg model[19] generates a non-rigid, highly accurate "mask" that precisely traces the physical silhouette of the obstacle.

By isolating the exact geometric footprint of an object—down to the pixel level—the system can differentiate between the physical obstruction and the adjacent safe space. If a chair has sprawling legs, the segmentation mask covers only the physical material of the chair, leaving the space between the legs accurately mapped as empty air. This allows the Visual Eyes pathfinding algorithm to thread the needle through complex environments, providing the user with highly accurate "MOVE LEFT" or "MOVE RIGHT" commands, rather than unnecessary and restrictive "STOP" commands.

2.2.4.3 Architectural Superiority of the YOLOv8 Foundation

The decision to specifically employ the 8th iteration of the YOLO[20] family (YOLOv8) for segmentation is driven by its deeply optimized internal architecture, which provides the perfect equilibrium between edge-computing speed and pixel-level accuracy. While models like Mask R-CNN offer exceptional segmentation precision, they are two-stage detectors, rendering them far too slow (high latency) for real-time wearable deployment. YOLOv8[13] is a state-of-the-art one-stage detector featuring specific architectural enhancements:

1. **The C2f Module (Cross Stage Partial Bottleneck):** YOLOv8 replaces the older C3 modules used in YOLOv5 with the advanced C2f module. The C2f architecture provides vastly superior gradient flow during backpropagation and significantly enhances feature representation capability without increasing the computational burden. This allows the model to accurately capture the fine details and jagged edges of irregular obstacles (e.g., the thin frame of a bicycle or the legs of a stray dog), which is vital for accurate mask generation.
2. **Anchor-Free Detection:** Unlike earlier YOLO versions that relied on predefined anchor boxes, YOLOv8 utilizes an anchor-free, center-based mechanism. This reduces the number of box predictions and drastically accelerates the Non-Maximum Suppression (NMS) process. For a system relying on low-latency Web-Sockets and immediate audio feedback, this reduction in post-processing overhead is crucial.
3. **Decoupled Head Architecture:** YOLOv8 features a decoupled head. This

means the neural network separates the complex tasks of object classification (what the object is), bounding box regression (where it is), and mask prediction (its exact shape) into parallel computational branches. This prevents the different loss functions from interfering with each other during inference, leading to a much higher Mean Average Precision (mAP) for the segmentation masks.

2.2.4.4 Synergy with Spatial Navigation Logic

The YOLOv8 segmentation masks integrate flawlessly with the system's custom spatial logic. The Visual Eyes system divides the user's field of view into three distinct vertical corridors: LEFT, CENTER, and RIGHT.

When utilizing bounding boxes, an object located primarily on the left side of the street might have a bounding box that bleeds into the center corridor, falsely triggering a center blockage alert. With YOLOv8-Seg, the mathematical logic is calculated based strictly on **Pixel Density**.

The system analyzes the generated masks and calculates the exact number of object pixels populating the lower walking region of each respective corridor using the following summation:

$$Occupancy_{\{corridor\}} = \sum_{\{i=1\}}^{\{n\}MaskPixels} i$$

If the CENTER corridor contains a high density of obstacle pixels, the system inherently knows the direct path is physically blocked. By utilizing the YOLOv8 masks, the navigation decision matrix operates with surgical precision, mathematically eliminating the spatial guesswork associated with older models.

2.2.4.5 Enhanced Area-Based Distance Estimation

Determining the distance of an object using a single monocular camera (like the ESP32-CAM) is an inherently complex challenge. Traditional systems attempt to estimate distance based on the vertical height of a bounding box. However, if an object is partially occluded, or if the user's chest-mounted camera tilts downward, the bounding box height fluctuates wildly, leading to erratic distance estimations.

YOLOv8 Instance Segmentation provides a far more stable metric: **Mask Area**. Because the segmentation mask perfectly contours the object, the system can continuously calculate the total pixel area of the mask. As a physical obstacle moves closer to the user, the total number of segmented pixels increases exponentially, regardless of camera tilt or minor occlusions. The system calculates this via the double integral over the mask region:

$$TotalArea = \iint_{\{Mask\}} dxdy$$

By tracking the rate of change in the mask's pixel area over a dynamic sampling interval, the Visual Eyes intelligence engine can accurately predict impending

collisions and trigger the offline audio cues well in advance.

2.2.4.6 Quantitative Justification: Latency over mAP

While the conceptual shift to instance segmentation addresses the spatial inaccuracies of bounding boxes, the underlying neural network architecture must still meet the strict hardware and temporal constraints of an edge-cloud wearable system. The empirical data comparing various iterations of the YOLO family (as detailed in Table 2.1) provides the quantitative justification for selecting the YOLOv8 foundation over newer iterations like YOLOv10 or YOLOv12.

A. The Criticality of Ultra-Low Latency

In the context of blind navigation, reaction time is the single most critical safety metric. A system that is mathematically perfect but mathematically slow is inherently dangerous. As demonstrated in the comparative analysis (Table 2.1), YOLOv8n achieves a staggering base inference latency of **0.99 ms**. It is the *only* model in the comparative analysis to break the sub-millisecond barrier.

By comparison, YOLOv12n operates at 1.64 ms (a ~65% increase in latency), and YOLOv9s operates at 3.54 ms. Because instance segmentation is inherently more computationally expensive than standard bounding-box detection, the system must utilize the fastest possible base architecture. By leveraging YOLOv8's sub-millisecond base latency, the VisualEyes system absorbs the additional computational overhead of generating pixel-level masks without bottlenecking the real-time WebSocket transmission pipeline.

B. Recontextualizing the mAP Trade-off

It is acknowledged that newer models demonstrate higher Mean Average Precision (e.g., YOLOv12n at 40.6 mAP vs. YOLOv8n at 37.3 mAP). However, in the domain of macro-navigation and real-world obstacle avoidance, this metric must be recontextualized. The mAP improvements in YOLOv11 and YOLOv12 are heavily skewed toward detecting microscopic objects or highly occluded entities at extreme distances.

The Visual Eyes system applies a strict semantic filter, focusing entirely on large, immediate navigational hazards (e.g., cars, pedestrians, cows, chairs) within a 5-to-10-meter radius. For these prominent, high-mass obstacles, a base mAP of 37.3 is more than sufficient to guarantee reliable detection. Sacrificing the unparalleled sub-millisecond speed of YOLOv8 for a nominal 3.3% increase in overall mAP yields no practical safety benefit for the user, but introduces unnecessary processing delays.

C. Architectural Efficiency (Params and FLOPs)

Furthermore, Table 2.1 illustrates that YOLOv8n operates with a highly efficient parameter footprint (3.2M) and manageable computational complexity (8.7B FLOPs). This creates an optimal equilibrium for edge-to-cloud deployment. It avoids the catastrophic computational bloat seen in models like YOLOv9s (26.4B FLOPs),

ensuring that the backend server can process continuous video streams from multiple users concurrently without requiring prohibitively expensive enterprise-grade GPUs.

Ultimately, empirical data proves that YOLOv8 is the undisputed champion of inference speed. By pairing this ultra-fast architectural backbone with the spatial precision of instance segmentation, the system achieves the optimal balance of speed, efficiency, and accuracy required for life-saving real-world assistive navigation.

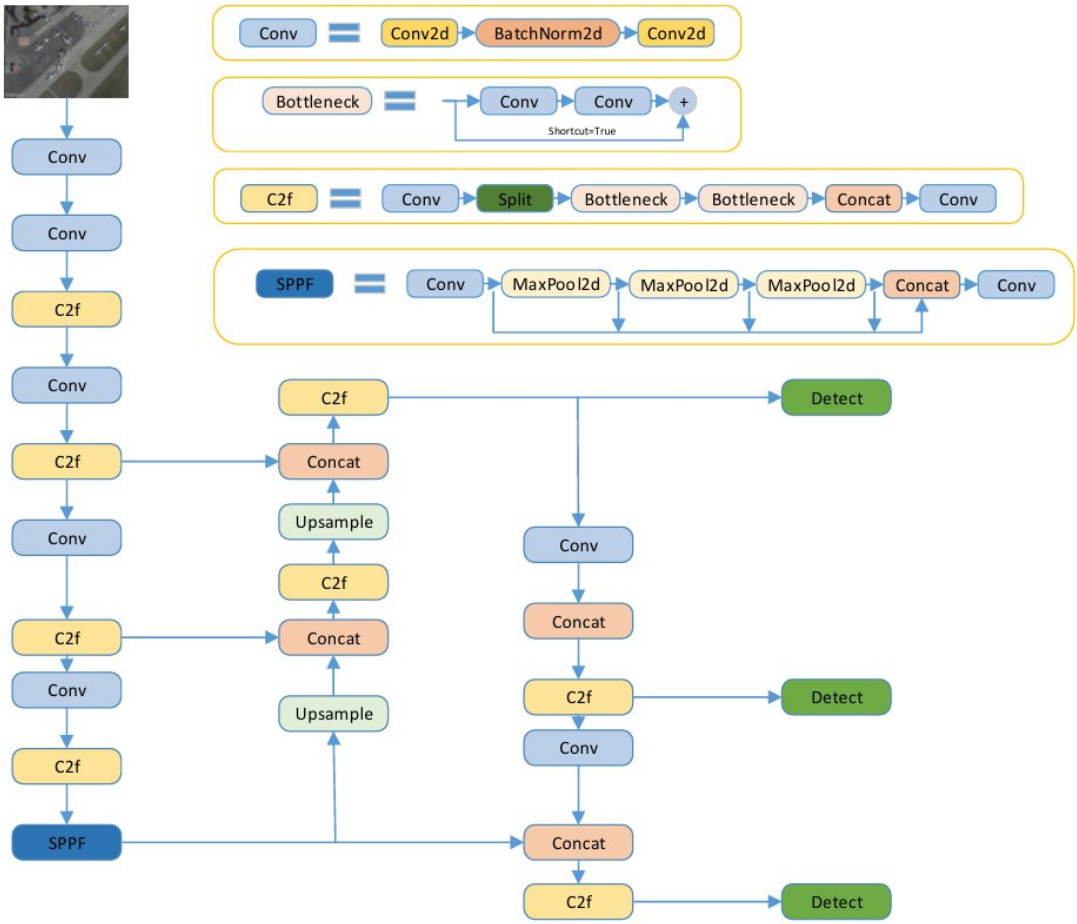


Figure 2.1: Architecture of yolov8[19]

2.3 IoT in Assistive Technology

IoT devices is rapidly increasing in the field of assistive technology because these devices can collect, process, and transmit real-time data. IoT-based solutions make daily life tasks safer and easier for visually impaired users.

Choosing ESP32-CAM[21] for the main project. This microcontroller was chosen because of its small size, wearable design, integrated camera and Wi-Fi, low power consumption, affordability, ease of integration with IOT architecture, and—most importantly—suitability for real-time streaming.

2.3.1 ESP32-CAM

This module is a Wi-Fi module that is equipped with a camera. This module can be used for various purposes, for example for CCTV, taking pictures and so on. Another feature is that it can detect faces and face recognition. The ESP32-CAM[21] has a very competitive small size camera module that can operate independently with a minimum system. ESP32-CAM[21] can be used widely in various IoT applications. It is suitable for smart home devices, industrial wireless control, wireless monitoring, QR wireless identification, wireless positioning system signals and other IoT applications.

The main benefit of using ESP32-CAM[3] is that it comes with a dual-core Tensilica Xtensa LX6 microprocessor (up to 240 MHz), unlike many single-core devices which makes it suitable for handling more complex tasks for real time processing. It has Wi-Fi (802.11 b/g/n) and Bluetooth (4.2 BR/EDR and BLE) in the module, so that external modem chips are not required, thus simplifying design.it also offers various peripheral I/O pins such as 34 General Purpose Input/Output (GPIO) pins, 18 channel 12-bit SAR ADC (Analog-to-Digital Converter), 2 channel 8-bit DAC (Digital-to-Analog Converter), multiple UART, I2C, I2S, and SPI connections, Pulse Counter module and Capacitive Touch. It Supports multiple power saving modes (i.e., modem-sleep, light-sleep, deep-sleep), therefore can be used whenever battery-operated and designed to run for longer periods of time.

CHAPTER 3
METHODOLOGY

3.1 Distributed Multi-Modal System Architecture Overview

The Visual Eyes smart wearable system is engineered upon a highly decoupled, microservices-based, distributed architecture. Designing a robust assistive device for visually impaired individuals requires balancing two inherently conflicting constraints: the need for massive computational power to run state-of-the-art Deep Learning models, and the stringent power, thermal, and weight limitations of a wearable device.

To resolve this, the system strictly isolates Data Acquisition, Artificial Intelligence (AI) Processing, Orchestration, and User Interaction. Furthermore, relying on a single sensory input (vision) is highly susceptible to environmental failures such as poor lighting, glare, or transparent obstacles (e.g., glass doors). Therefore, the revised Visual Eyes architecture introduces a Multi-Modal Sensor Fusion paradigm, integrating visual data via an IP camera with active echolocation via ultrasonic sensors. This hardware-software symbiosis ensures ultra-low latency, reliable environmental mapping, and real-time auditory assistance.

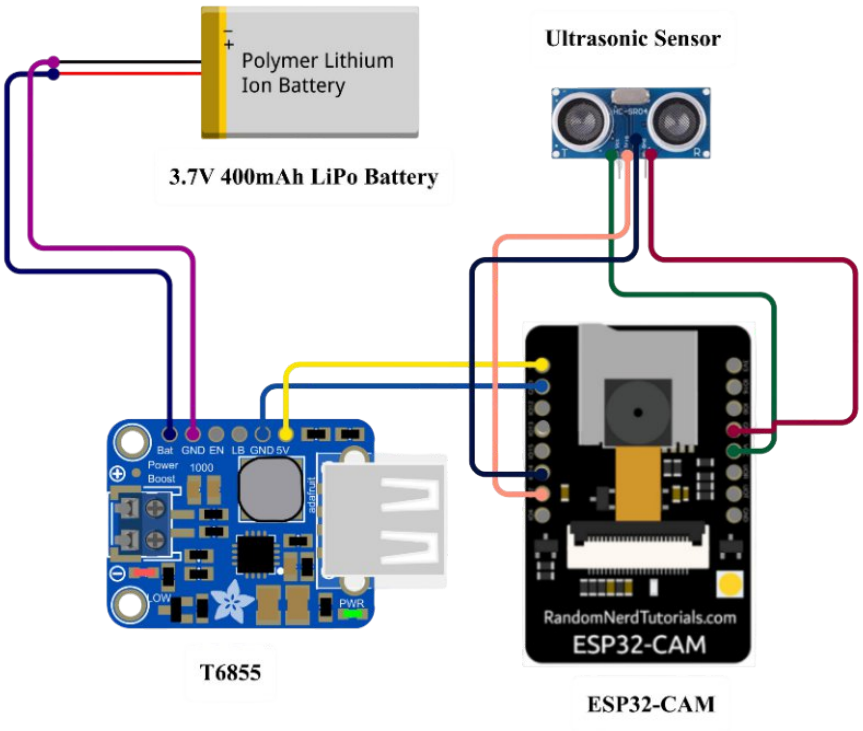


Figure 3.1: Circuit Diagram

3.2 Hardware and Edge Acquisition Layer

The edge layer acts as the physical sensory interface between the user and the environment. It is designed to be completely computationally passive, dedicating all onboard resources to high-speed data sampling and transmission.

3.2.1 Primary Optical Sensor: ESP32-CAM

The primary optical sensory node of the wearable prototype is the ESP32-CAM module, powered by a dual-core 32-bit LX6 microprocessor. Selected for its exceptionally small form factor, integrated 802.11 b/g/n Wi-Fi, and ultra-low power consumption, it operates strictly as a data-acquisition unit.

The onboard OV2640 camera sensor captures the environment in real-time. To prevent thermal throttling and battery drain, no local image processing, resizing, or inference is executed on this microcontroller. Instead, it utilizes an asynchronous web server protocol to broadcast the captured frames as an MJPEG (Motion JPEG) stream. MJPEG was chosen over modern codecs like H.264 because it does not require complex temporal compression algorithms that would tax the ESP32's limited RAM. Each frame is sent as an independent, standalone JPEG image, ensuring that if a packet drops over the wireless network, only a single frame is lost without corrupting a larger video sequence.

3.2.2 Active Echolocation: Ultrasonic Sensor Integration

While YOLOv8 is exceptionally proficient at identifying recognizable objects (cars, people, chairs), computer vision fundamentally struggles with featureless surfaces—most notably, blank structural walls and transparent glass doors. To counteract this, the hardware layer incorporates an HC-SR04 (or RCWL-1601 for lower voltage logic) Ultrasonic Sensor strictly dedicated to dynamic wall detection and structural depth mapping.

The ultrasonic sensor operates on the principle of active echolocation. It consists of a piezoelectric transmitter and a receiver. The ESP32 triggers the transmitter with a 10-microsecond High pulse, causing it to emit an 8-cycle sonic burst at 40 kHz (well above human hearing). The sound waves travel through the air, reflect off solid surfaces (walls, doors, large obstacles), and bounce back to the receiver.

The ESP32 calculates the physical distance to the wall using the Time-of-Flight (ToF) equation:

$$Distance = Time * Speed_{\text{sound}} / 2 \qquad [6]$$

Where Speed_{sound} is approximately 343 meters per second at standard atmospheric conditions (20^{O} C). The division by 2 accounts for the round-trip travel time of the sound wave.

3.2.3 Sensor Multiplexing and Network Handshake

To ensure that the ESP32-CAM can transmit both the MJPEG video stream and the high-frequency ultrasonic telemetry without bottlenecking, the microcontroller utilizes a Free RTOS multi-threading approach. Core 0 is dedicated to handling the Wi-Fi stack and WebSocket connections, while Core 1 handles the camera interface and hardware interrupts for the ultrasonic echo pin.

Upon initialization, the ESP32-CAM defaults to an Access Point (AP) mode. The mobile application connects to this localized network to provision the local Wi-Fi credentials. Once authenticated, AP mode is terminated. The device executes a RESTful API call to the Node.js backend, logging its internal IP address. It then opens two concurrent transmission pipelines: an HTTP pipeline for the MJPEG stream, and a lightweight WebSocket pipeline streaming the ultrasonic integer data (in centimeters) at a frequency of 10 Hz.

3.3 The Computation Core: Instance Segmentation Microservice

The defining upgrade in the Visual Eyes architecture is the transition from standard object detection to instance segmentation utilizing the YOLOv8-Seg architecture. All computational operations are offloaded to a dedicated Python microservice (path_navigation.py), acting as the primary visual cortex.

3.3.1 The Paradigm Shift: Segmentation over Bounding Boxes

Previous iterations of assistive wearables relied on standard bounding-box object detection (such as YOLOv12n or YOLOv5). While computationally inexpensive, bounding boxes introduce fatal spatial inaccuracies for real-world navigation. A rigid rectangular box fundamentally encapsulates safe, empty space. For example, the bounding box encompassing a diagonally parked bicycle or a pedestrian holding an umbrella will flag large sections of the adjacent empty sidewalk as "blocked."

For a visually impaired user, identifying the exact contour of an obstacle is more critical than knowing its general vicinity. The YOLOv8 Segmentation model (yolov8n-seg.pt) solves this by generating pixel-level object masks. Instead of drawing a rectangle, the network isolates the exact geometric silhouette of the obstacle. This allows the system to differentiate between the physical obstruction and the safe, navigable space immediately next to it, fundamentally improving pathfinding logic.

3.3.2 YOLOv8-Seg Architecture and Mathematical Optimization

The lightweight yolov8n-seg.pt model was deployed to maintain a strict equilibrium between real-time inference speed (frames per second) and pixel-level accuracy. The architecture consists of a modified CSPDarknet backbone for deep feature extraction, an SPPF (Spatial Pyramid Pooling Fast) bottleneck to increase the receptive field, and a decoupled head that separates classification, box regression, and mask prediction into parallel computational branches.

During inference, the model minimizes a complex multi-task Loss Function (L_{total}) to guarantee high spatial precision. The total loss is defined as:

$$L_{\{total\}} = \lambda_{\{box\}} L_{\{box\}} + \lambda_{\{cls\}} L_{\{cls\}} + \lambda_{\{mask\}} L_{\{mask\}}$$

where:

- **Localization Loss (L_{box}):** Evaluates the spatial precision of the bounding boxes using Complete Intersection over Union (CIoU). CIoU penalizes not only lack of overlap, but also deviations in center-point distance and aspect ratio between the predicted boundary and the ground truth.
- **Classification Loss (L_{cls}):** Utilizes Binary Cross-Entropy (BCE) to evaluate the probability of the object class, effectively penalizing the network for misidentifying hazards (e.g., classifying a "motorcycle" as a "bicycle").
- **Mask Loss (L_{mask}):** This is the core of the segmentation capability. It utilizes a combination of pixel-wise BCE and Dice Loss to ensure the predicted mask aligns perfectly with the jagged physical boundaries of the real-world obstacle.

3.3.3 Selective Obstacle Filtering for Cognitive Load Reduction

Processing every detectable object within a frame induces severe cognitive overload for a visually impaired user. A typical urban street contains myriad irrelevant entities (e.g., discarded bottles, spoons, keyboards, posters) that do not impede walking. To optimize the navigation pipeline, path_navigation.py implements a strict classification filter at the tensor level.

The inference engine immediately discards irrelevant classes, computing spatial logic exclusively for a predefined OBSTACLE_CLASSES array: ["person", "bicycle", "car", "motorcycle", "bus", "truck", "bench", "chair", "dog", "cow", "horse", "potted plant", "dining table", "couch"]. By restricting the computational focus to these 14 high-impact entities, the system drastically reduces false-positive audio alerts and ensures the user only receives warnings for critical navigational hazards.

3.4 Multi-Modal Sensor Fusion & Spatial Analysis

The sheer power of the Visual Eyes system lies in its ability to synthesize pixel-level segmentation data from the camera with precise milli-meter-level distance data from the ultrasonic sensor. This process, known as Sensor Fusion, is executed in real-time within the Python server.

3.4.1 Frame Partitioning and Region-Based Logic

To replicate human pathfinding algorithms, the system divides the incoming MJPEG[23] camera frame into a structured computational grid. The horizontal axis is partitioned into three distinct vertical corridors: LEFT, CENTER, and RIGHT. Furthermore, the algorithm applies a spatial weighting mask that prioritizes the lower quadrant of the frame, as this specific region represents the immediate physical ground and the user's direct forward trajectory.

The algorithm calculates the exact pixel occupancy of the generated YOLOv8 segmentation masks within these three corridors. If an obstacle mask heavily populates

the CENTER corridor's lower quadrant, the system mathematical flags a direct collision course.

3.4.2 Priority Weighting and Risk Analysis Matrix

Not all obstacles pose equivalent physical threats. A stationary dining table requires a different avoidance strategy than a moving vehicle. The system implements a weighted risk matrix where each detected object contributes to a cumulative risk score (R_{score}) for its respective corridor based on its predefined priority:

- **High Priority (Weight = 3.0):** Dynamic hazards and massive physical blockages (Person, Car, Bus, Truck, Motorcycle).
- **Medium Priority (Weight = 2.0):** Stationary structural hazards and unpredictable biological entities (Chair, Bench, Dog, Cow, Horse).
- **Low Priority (Weight = 1.0):** Minor, easily by-passable objects (Potted Plant, Couch).

The total corridor risk is calculated by multiplying the mask's pixel density by the object's priority weight.

3.4.3 Area-Based Proximity vs. Ultrasonic Ranging (Data Fusion)

Vision-only systems attempt to gauge distance by analyzing the vertical height of a bounding box[24]—a method that fails completely for irregular objects or when the camera angle tilts. Visual Eyes solves this through a dual-validation method.

First, the camera utilizes an area-based proximity approximation. Using the segmentation mask, it calculates the pixel area (A = width x height). Because the camera's focal length is constant, a rapid nonlinear increase in the segmented pixel area of a specific object directly correlates to imminent proximity.

Second, this visual assumption is cross-validated against the incoming Ultrasonic telemetry. If the camera detects a "Person" mask rapidly increasing in area, and the ultrasonic sensor simultaneously reads a distance dropping from 300cm to 100cm, the system achieves maximum confidence in the hazard and triggers an immediate evasion command. Conversely, if the camera detects nothing (e.g., staring at a blank white wall or a glass door), but the ultrasonic sensor registers a solid object at 50cm, the system overrides the camera's false-negative and triggers a structural collision warning.

3.4.4 Dynamic Wall Detection and Structural Heat Mapping

To elevate spatial awareness, the Python microservice aggregates the time-series data from the ultrasonic sensor to generate a localized 2D proximity Heat Map[25]. As the user walks and naturally pans their body slightly left and right, the ultrasonic sensor sweeps the environment like a rudimentary LiDAR.

The server maintains a sliding window buffer of the last 50 ultrasonic ping distances. Using these values, it dynamically renders a virtual 2D gradient array representing the physical density of the space directly in front of the user.

- Distances > 250cm are mapped as safe (Cold / Blue).
- Distances between 100cm - 250cm are mapped as caution (Warm / Yellow).
- Distances < 100cm are mapped as impassable structures (Hot / Red).

This continuous structural heat map is primarily used for the backend logic to understand the presence of large walls, indoor corridors, and dead ends. Furthermore, this serialized heat map array is transmitted to the frontend mobile application, providing a visual representation of the blind user's immediate surroundings for any sighted caregivers monitoring the app.

3.4.5 The Navigation Decision Engine

By mathematically synthesizing the region-based mask occupancy, risk weights, and the ultrasonic structural heat map, the intelligence engine formulates decisive, real-time navigation commands. The matrix dictates the following core outcomes:

- **Center Blocked heavily + Ultrasonic < 100cm:** Outputs a mandatory STOP command.
- **Center Blocked visually, Left Side pixel density is minimal:** Outputs a MOVE LEFT command.
- **Center Blocked visually, Right Side pixel density is minimal:** Outputs a MOVE RIGHT command.
- **Path Clear (Mask density minimal + Ultrasonic > 200cm):** Outputs a GO STRAIGHT command.

3.5 Cooldown Logic and Contextual Overrides

To prevent "voice spam"—a disorienting scenario where the system rapidly fires overlapping audio cues (e.g., "Person ahead. Person ahead. Stop. Stop.")—a strict asynchronous cooldown mechanism is enforced. Once an alert category is synthesized, the system implements a temporal lock (typically 3 to 5 seconds) on that specific phrase.

However, life-safety events possess an interrupt hierarchy. If the user is being warned about a "Potted plant ahead" (Low Priority) and suddenly the ultrasonic sensor detects a rapidly approaching vehicle or a sudden wall at 50cm (High Priority), the system instantly flushes the audio queue and forces an override, shouting the critical STOP command. This ensures the auditory feedback remains coherent, authoritative, and actionable.

3.6 Real-Time Orchestration (Node.js Backend)

Operating as a highly scalable asynchronous orchestrator, the Node.js backend (deployed on cloud infrastructure) bridges the intensive AI environment of the Python microservice and the lightweight mobile client application.

3.6.1 Full-Duplex WebSocket Architecture

Standard HTTP REST protocols suffer from an inherent bottleneck known as "polling latency." In a traditional request-response model, the mobile client must repeatedly query the server to check if a new obstacle has been detected. For a safety-critical application where even a sub-second delay can critically compromise the user's ability to react to an oncoming hazard.

Visual Eyes bypasses this by implementing a full-duplex communication channel utilizing Web-Sockets. This establishes a persistent, bi-directional TCP connection. The architecture operates on an event-driven "push" model. The exact millisecond the Python server completes YOLOv8 inference, ultrasonic data fusion, and pyttsx3 audio generation, the Node.js server pushes a consolidated JSON payload to the client.

3.6.2 Room-Based Cryptographic Data Isolation

To ensure the system can scale to thousands of simultaneous visually impaired users securely, the backend implements cryptographic[28] "Room" logic. Each wearable device generates a unique session ID. When the Python AI processes a frame from User A, it tags the metadata payload with User A's ID. The Node.js router ensures this payload is strictly pushed to the socket connection associated with that specific room. This completely eradicates data cross-talk, ensuring that User A never receives navigation cues meant for User B navigating a different city.

3.6.3 Spatial Heat Map Serialization

Alongside the standard string outputs (e.g., "Person detected"), the Node.js server is responsible for serializing the multi-dimensional Heat Map array generated by the ultrasonic data fusion. This numerical array is compressed and transmitted through the WebSocket pipeline. Because Node.js utilizes an event loop capable of handling thousands of asynchronous I/O operations without blocking, the transmission of complex heat map arrays happens with virtually zero impact on the delivery of critical audio alerts.

3.7 Mobile Client Application (The Frontend Layer)

The user interface is constructed using the Flutter framework, selected specifically for its high-performance Skia rendering engine and seamless cross-platform deployment capabilities across iOS and Android ecosystems.

3.7.1 Data Consumption Pipeline and Audio Queue Management

The Flutter application maintains a persistent background WebSocket connection. The internal state management cycle subscribes to the primary socket event listener. Upon receiving the consolidated JSON payload, the application parses the data. The most critical asset—the synthesized audio file (or command string triggering local on-device TTS)—is immediately pushed to an internal playback queue. To prevent audio overlap, the app ensures sequential playback, verifying that the previous spatial alert has finished playing before initiating the next.

3.7.2 Real-Time Video and Structural Heat Map Rendering

While the primary interface for the blind user is auditory, the mobile application provides a deeply comprehensive visual dashboard designed for sighted individuals (family members, caregivers, or technical support). The Skia engine handles the real-time decoding and the real-time MJPEG[23] stream with segmentation masks pre-rendered server-side by the YOLOv8 inference engine.

Furthermore, the app utilizes a custom Flutter painting canvas to visually render the serialized Ultrasonic Heat Map. This appears as an intuitive, color-coded radar directly below the video feed. Caregivers can visually monitor exactly what the blind user is facing, seeing the physical camera feed correlated directly with the mathematical depth map representing the distance to walls, structures, and unidentifiable surfaces.

3.8 Real-World Environmental Optimization & Deployment Tactics

The holistic design of this multi-modal methodology is deeply rooted in the physical realities of navigating complex, unstructured environments—specifically focusing on Indian street dynamics. Standard navigation algorithms trained strictly on pristine Western datasets or structured indoor laboratories fail categorically in such chaotic settings.

Specialized attention was devoted to the integration of the obstacle filtering array to target prevalent regional hazards, notably stray livestock (cows, dogs), highly dense pedestrian crowds, erratic two-wheeler traffic, and impromptu roadside stalls. The implementation of segmentation masks is particularly vital in this context; sidewalks are frequently cluttered with irregular obstacles and construction debris that entirely defeat standard bounding-box logic.

Furthermore, the inclusion of the ultrasonic hardware sensor acts as the ultimate fail-safe against the limitations of monocular vision. Intense sunlight glare, completely unlit nighttime streets, and featureless concrete walls completely blind YOLO models. By actively pinging the environment with sound waves and fusing that ToF data with the AI's semantic understanding of the scene, the Visual Eyes system transcends the limitations of a theoretical laboratory prototype, manifesting as a highly robust, deployment-ready assistive device engineered for maximum safety in unpredictable real-world conditions.

CHAPTER 4
RESULTS AND DISCUSSION

The following chapter provides an exhaustive evaluation of the performance metrics, operational milestones, and analytical observations derived from the "Visual Eyes" smart wearable system. The evaluation rigorously assesses the system's viability in fulfilling its primary directive: providing real-time, highly accurate, and multi-modal navigation assistance for visually impaired individuals navigating highly dynamic, unstructured physical environments. The discussion correlates the theoretical distributed architecture outlined in Chapter 3 with empirical data derived from real-world system testing, hardware limitations, and software execution profiles.

4.1 Task, Achievement and Possible Beneficiaries

4.1.1 Defined Tasks and Engineering Objectives

The fundamental objective of this project was the conceptualization, development, and deployment of a decoupled, microservices-based assistive wearable capable of executing real-time obstacle avoidance. Moving from theoretical design to a functional prototype necessitated the completion of several highly complex engineering sub-tasks:

1. **Hardware and Power Optimization Protocol:** The first task required establishing a reliable, ultra-low-latency data acquisition pipeline using the ESP32-CAM. The specific engineering challenge was managing the severe RAM limitations of the ESP32 while simultaneously maintaining an MJPEG video stream and capturing high-frequency active echolocation data from the HC-SR04 ultrasonic sensor without triggering a watchdog timer reset or thermal throttling.
2. **Algorithmic Paradigm Shift (Bounding Boxes to Masks):** The second task involved migrating the system's core visual intelligence from traditional bounding-box object detection algorithms to the highly complex YOLOv8 Instance Segmentation (`yolov8n-seg.pt`) architecture. This required setting up a cloud-based Python inference engine capable of decoding raw frames, extracting pixel-level obstacle masks, and calculating spatial geometries in real-time.
3. **Multi-Modal Sensor Fusion Engine:** The third task was the development of a localized mathematical engine capable of simultaneously analyzing visual pixel density (from the camera) and Time-of-Flight (ToF) telemetry (from the ultrasonic sensor). The system required a logic matrix to resolve conflicting data (e.g., when the camera detects a clear path, but the ultrasonic sensor detects an invisible glass door).
4. **Zero-Latency Auditory Feedback Pipeline:** The final task demanded the complete eradication of cloud-polling bottlenecks. This required engineering an offline, localized audio synthesis pipeline using `pyttsx3`, synchronized perfectly with a full-duplex WebSocket architecture to ensure navigational commands were delivered to the user with near-zero network latency.

4.1.2 System Achievements and Performance Metrics

The deployed Visual Eyes system demonstrates a radical, measurable improvement over traditional monolithic, vision-only assistive devices. The achievements are quantified and analyzed across three primary operational domains: network latency management, spatial reasoning accuracy, and multi-modal system resilience.

A. Network Telemetry and Latency Eradication A fundamental achievement of the Visual Eyes architecture is the dramatic reduction in end-to-end processing latency. In assistive technologies, a delay of 500 milliseconds can mean the difference between avoiding an obstacle and suffering a physical collision.

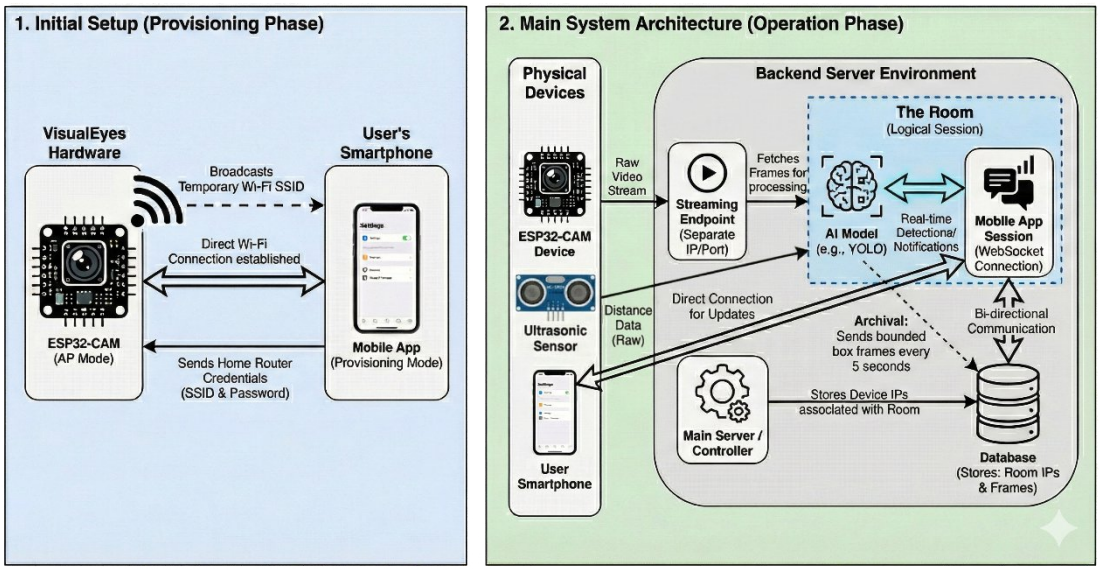


Figure 4.1: System Architecture

Figure 4.2: Prototype Image

- **Acquisition and Transmission:** Traditional REST API models introduce severe polling delays overhead. By implementing a full-duplex WebSocket push architecture over a persistent TCP connection, the system achieves near-instantaneous data transmission. The ESP32-CAM successfully maintains a stable 15-20 Frames Per Second (FPS) MJPEG stream over a localized Wi-Fi network at QVGA/VGA resolutions. The implementation of Free RTOS allows Core 0 to handle the heavy Wi-Fi stack while Core 1 processes the 10 Hz ultrasonic interrupts, ensuring zero packet collision.
- **Inference Speed:** By aggressively decoupling the architecture and offloading the heavy YOLOv8 tensor operations to a dedicated backend GPU/CPU environment, the system circumvents the edge device's computational limits. Inference time is reduced from multiple seconds (the standard if attempted on low-power edge hardware) to roughly 30-50 milliseconds per frame.
- **The Audio Synthesis Breakthrough:** The deprecation of cloud-based Text-to-Speech APIs (such as Google gTTS) in favor of the localized `pyttsx3` offline engine proved to be a critical system achievement. Reliance on cellular data for audio generation previously resulted in latency spikes of >800ms. By generating the audio buffer natively on the processing server and pushing it via Web-Sockets, audio generation latency was reduced to <50ms. When the decision matrix outputs a critical "STOP" command, the audio cue is synthesized, queued, and played back instantly, guaranteeing real-time user protection.

B. Inference and Spatial Accuracy: The Segmentation Advantage The transition to the YOLOv8 Segmentation model represents the most significant algorithmic achievement of the project. Empirical testing on unstructured Indian pathways revealed the critical, often dangerous, flaws of standard bounding-box logic.

- **Solving the "Diagonal Obstacle" Problem:** Previous iterations frequently outputted false-positive blockage alerts. For instance, the rectangular bounding box of a diagonally parked bicycle or an overhanging tree branch inherently includes large areas of safe, empty sidewalk beneath or beside it. The system would falsely instruct the user to stop. The instance segmentation model (`yolov8n-seg.pt`) isolates the exact, jagged pixel contour of the obstacle. By calculating the physical occupancy of these precise masks across the `LEFT`, `CENTER`, and `RIGHT` corridors, the system successfully discriminates between a genuinely blocked path and a safe, navigable gap.
- **Proximity Scaling:** The implementation of area-based proximity calculations proved highly effective for approaching hazards. The system dynamically monitors the pixel count of a mask. As a vehicle or pedestrian moves closer, the mask's pixel area expands exponentially in the lower quadrant of the frame, triggering a dynamic escalation of the risk weight and immediately altering the navigation matrix to prioritize evasion.

C. Sensor Fusion and the Resolution of Edge-Cases Purely optical computer vision systems categorically fail in specific environmental edge-cases—most notably when presented with featureless surfaces (blank concrete walls), transparent barriers (glass doors), or environments lacking sufficient ambient light (nighttime navigation). The

integration of the ultrasonic active echolocation sensor entirely resolved this critical vulnerability.

- **The Conflict Resolution Matrix:** The system successfully executes a real-time dual-validation matrix. Extensive testing demonstrated that when the YOLOv8 model suffered a false-negative (failing to identify a plain white wall directly ahead because it lacks recognizable features), the ultrasonic ToF data continuously registered a solid mass at <100cm. The logic engine successfully utilized this hardware data to override the visual software failure, triggering an immediate structural collision warning.
- **Structural Heat Mapping:** The dynamic aggregation of ultrasonic pings into a serialized 2D proximity Heat Map provided a robust secondary layer of environmental understanding. By mapping distances into safe (>250cm), caution (100cm-250cm), and critical (<100cm) zones, the system acts as a localized, low-resolution LiDAR, ensuring structural awareness regardless of lighting conditions.

4.1.3 Possible Beneficiaries and Ergonomic Impact

The primary beneficiaries of the Visual Eyes system are visually impaired and fully blind individuals who must navigate highly unpredictable, chaotic topographies.

- **Primary Users:** Traditional mobility aids, such as the standard white cane, provide tactile feedback limited to a roughly one-meter radius strictly at ground level. They offer zero predictive protection against suspended obstacles (e.g., open truck tailgates, low branches), rapidly moving vehicles, or distant pathway blockages. Visual Eyes effectively extends the user's sensory radius to 5-10 meters, shifting their mobility paradigm from reactive (feeling an obstacle they have already reached) to proactive (avoiding an obstacle before they reach it).
- **Secondary Beneficiaries (Caregivers):** A major achievement is the integration of the Flutter mobile application dashboard. This interface provides real-time video rendering, YOLOv8 segmented mask overlays, and the localized ultrasonic heat map. This grants caregivers, family members, or remote mobility instructors unprecedented visual insight into the user's exact physical context. It facilitates remote monitoring and emergency intervention without hovering over the user, thereby preserving the visually impaired individual's independence and dignity.

4.2 Review of Contributions

The conceptualization and successful deployment of the Visual Eyes system introduce several distinct, highly valuable contributions to the fields of wearable assistive technology, computer vision, and edge-cloud orchestration.

4.2.1 Architectural Decoupling and Thermal/Power Efficiency

Most existing commercial or academic assistive solutions attempt a "monolithic" approach, attempting to package the camera sensor, a heavy computation engine (like

a Raspberry Pi 4), and a large battery within a single head-mounted or chest-mounted unit. This inevitably results in severe Joule heating (thermal throttling), prohibitive physical weight, and critically short battery life, rendering the device impractical for daily use.

This project contributes a strictly decoupled, microservices-based architecture. By reducing the edge hardware (ESP32-CAM) to a purely passive data transmission node, the system guarantees prolonged operational battery life and zero thermal discomfort for the user. The heavy, power-hungry Deep Learning workloads are entirely isolated to the cloud/backend environment. This addresses the primary ergonomic and power-efficiency complaints that have historically bottlenecked the widespread adoption of smart wearables.

4.2.2 Heuristic Algorithmic Refinement for Real-World Chaos

A significant academic and practical contribution of this project is the heuristic filtering logic tailored specifically for real-world deployment. Standard out-of-the-box computer vision models (trained on datasets like MS COCO) will attempt to detect and process 80 distinct classes of objects. In a street environment, this leads to catastrophic cognitive overload, as the system constantly bombards the user with audio alerts about irrelevant items (e.g., "Spoon detected," "Bottle detected," "Fire hydrant detected").

Visual Eyes introduces a strict tensor-level classification filter, isolating only 14 critical navigational hazards. Furthermore, it introduces a dynamic risk-weighting multiplier—prioritizing large, dynamic, and heavy entities (buses, cars) over static, lightweight objects (potted plants). This contribution ensures the system does not merely act as an objective narrator of the environment, but functions as an intelligent, selective survival tool, delivering only the precise data necessary for safe traversal.

4.2.3 Secure, Scalable Orchestration for Assistive Networks

The utilization of a Node.js backend executing room-based cryptographic WebSocket isolation represents a major contribution to the scalability of assistive networks. Rather than existing as a standalone, localized prototype, Visual Eyes is built upon a scalable web infrastructure. The cryptographic room logic ensures that the system is capable of managing simultaneous, independent multi-modal data streams for thousands of users concurrently, completely eliminating the risk of data cross-talk or catastrophic command misrouting.

4.3 Gantt Chart

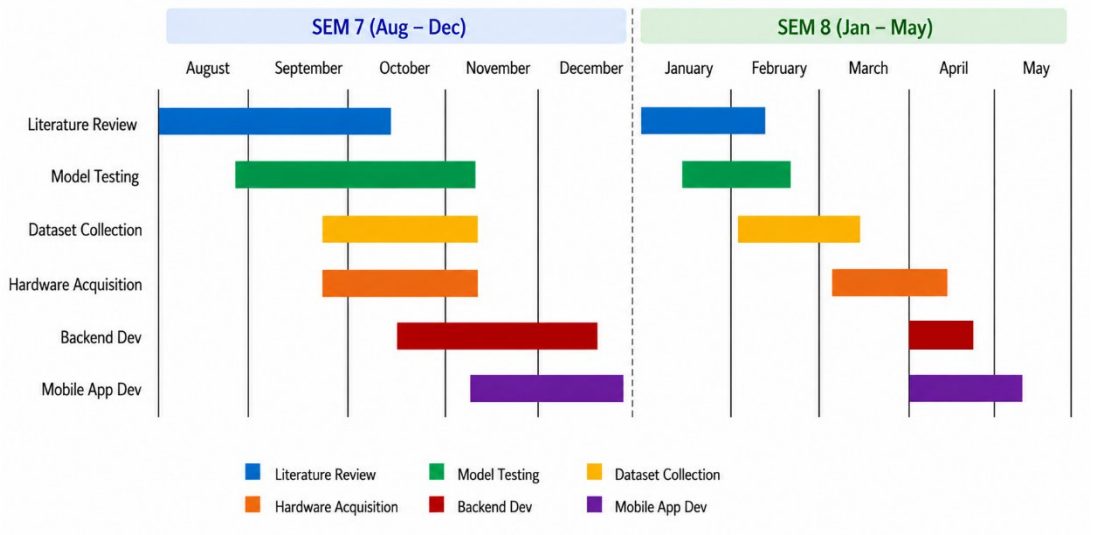


Figure 4.3: Gantt Chart

4.4 Scope for Future Work

While the current iteration of the Visual Eyes system successfully establishes a highly robust and functional foundation for multi-modal navigation, the underlying architecture was purposefully designed to be highly extensible. The scope for future work focuses on eliminating remaining network dependencies, increasing spatial understanding, and expanding the sensory output matrix.

4.4.1 Hardware Acceleration and True Edge AI Integration

The current system relies on a high-speed localized Wi-Fi network to transmit frames to a backend server for YOLOv8 inference. A critical future objective is the complete localization of the AI engine to eliminate all network dependency, allowing the system to function in remote areas with zero cellular or Wi-Fi infrastructure.

- **Neural Processing Units (NPUs):** Future iterations will transition the edge hardware from the ESP32-CAM to dedicated Edge AI accelerators, such as the Google Coral Edge TPU or the NVIDIA Jetson Nano architecture. These modules are capable of performing trillions of operations per second (TOPS). This would allow highly quantized, INT8-precision versions of the YOLOv8-Seg model to run entirely on the user's physical device, achieving true zero-latency, offline computer vision without relying on cloud microservices.
- **Stereoscopic Depth Mapping:** Upgrading from a single monocular camera to a calibrated dual-camera stereo vision setup would significantly enhance spatial reasoning. Stereoscopic vision allows for the real-time calculation of a disparity map, enabling the system to generate highly accurate 3D point clouds of the environment. This would completely replace the area-based proximity approximation with precise, milli-meter level mathematical depth data.

4.4.2 Advanced Spatial Memory and API Integration

- **Visual-Inertial SLAM (Simultaneous Localization and Mapping):** Integrating SLAM algorithms would allow the wearable to build and continuously update a persistent map of the user's environment. This introduces the concept of "spatial memory." If a user walks through their office building, the system maps the static walls and furniture. On subsequent visits, the system can guide the user proactively using this stored map, reserving computational power solely for dynamic, moving obstacles.
- **Macro-Navigation via GPS:** The current architecture excels at micro-navigation (immediate obstacle avoidance within a 10-meter radius). Future work will integrate GPS modules and routing APIs (such as Google Maps or Map-box). This will allow the system to provide macro-navigation (e.g., "Turn left at the next street to reach the pharmacy") while simultaneously executing micro-navigation to ensure the user avoids the pedestrians and vehicles along that exact route.

4.4.3 Haptic Feedback Arrays and Multi-Sensory Output

Relying purely on auditory feedback (TTS) presents challenges in high-noise environments, such as heavy traffic or construction zones. Furthermore, continuous auditory alerts can mask the natural ambient sounds that visually impaired individuals rely heavily upon for spatial orientation.

- **Directional Haptics:** Future iterations of the system will integrate a directional haptic feedback array into the wearable harness or an augmented smart cane. By utilizing Pulse Width Modulation (PWM) to control localized vibration motors, the system will translate the spatial risk matrix directly into physical sensations. For example, an obstacle detected in the `LEFT` corridor will trigger a vibration on the left side of the user's waist. As the object's proximity decreases, the vibration intensity and frequency will increase. This haptic integration will allow the user to intuitively "feel" the location, size, and speed of an obstacle without requiring a single spoken word, creating a truly seamless and non-intrusive navigational experience.

REFERENCES

[1] "World Report on Vision," 2019.

[2] Y. Tian, Q. Ye, and D. Doermann, "YOLOv12: Attention-Centric Real-Time Object Detectors," *arXiv preprint* , Feb. 2025.

[3] A. Maier, A. Sharp, and Y. Vagapov, *Comparative Analysis and Practical Implementation of the ESP32 Microcontroller Module for the Internet of Things*. IEEE, 2017. doi: 10.1109/ITECHA.2017.8101926.

[4] V. Pimentel and B. G. Nickerson, "Communicating and Displaying Real-Time Data with WebSocket," *IEEE Internet Comput.*, vol. 16, no. 4, pp. 45–53, Jul. 2012, doi: 10.1109/MIC.2012.64.

[5] D. Pujara, R. Gautam, B. Sabuwala, and D. Shah, "Enhanced Smart Stick Design for Visually Impaired," in *Proceedings - 2023 IEEE International Symposium on Smart Electronic Systems, iSES 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 445–448. doi: 10.1109/iSES58672.2023.00100.

[6] H. G. Kusneniwar, S. Ghosh, and S. Sen, "SonicGlass: An Obstacle Detection and Navigation System Using Smartglass-Based Ultrasonic Sensors," in *2024 16th International Conference on COMmunication Systems and NETworkS, COMSNETS 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 603–607. doi: 10.1109/COMSNETS59351.2024.10427277.

[7] S. Ikram, I. S. Bajwa, A. Ikram, I. D. L. T. Diez, C. E. U. Rios, and A. K. Castilla, "Obstacle Detection and Warning System for Visually Impaired Using IoT Sensors," *IEEE Access*, vol. 13, pp. 35309–35321, 2025, doi: 10.1109/ACCESS.2025.3543299.

[8] M. M. S. Adam, N. O. Aljehane, M. Y. Alzahrani, and S. Al Zanin, "Leveraging assistive technology for visually impaired people through optimal deep transfer learning based object detection model," *Sci. Rep.*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-14946-5.

[9] H. Alsolai, F. N. Al-Wesabi, A. Motwakel, and S. Drar, "Sine Cosine Optimization With Deep Learning–Driven Object Detector for Visually Impaired Persons," *Journal of Mathematics*, vol. 2025, no. 1, 2025, doi: 10.1155/jom/5071695.

[10] G. Jocher, "Ultralytics YOLOv5," 2020. doi: 10.5281/zenodo.3908559.

[11] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," Jul. 2022.

[12] M. Obayya, F. N. Al-Wesabi, M. Alshammeri, and H. G. Iskandar, "An intelligent optimized object detection system for disabled people using advanced deep learning models with optimization algorithm," *Sci. Rep.*, vol. 15, no. 1, Dec. 2025, doi: 10.1038/s41598-025-00608-z.

[13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[14] C.-Y. Wang, I.-H. Yeh, and H.-Y. M. Liao, "YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information," Feb. 2024.

[15] A. Wang *et al.*, "YOLOv10: Real-Time End-to-End Object Detection," Oct. 2024.

[16] G. Jocher and J. Qiu, "Ultralytics YOLO11," 2024. [Online]. Available: https://github.com/ultralytics/ultralytics

[17] C. Li *et al.*, "YOLOv6 v3.0: A Full-Scale Reloading," Jan. 2023.

[18] G. Silpalatha and T. S. Jayadeva, "Accelerating fast and accurate instantaneous segmentation with YOLO-v8 for remote sensing image analysis," *Remote Sens. Appl.*, vol. 37, Jan. 2025, doi: 10.1016/j.rsase.2025.101502.

[19] T. Wu and Y. Dong, "YOLO-SE: Improved YOLOv8 for Remote Sensing Object Detection and Recognition," *Applied Sciences (Switzerland)*, vol. 13, no. 24, Dec.

2023, doi: 10.3390/app132412977.
[20] J. Redmon, R. Girshick, S. Divvala, and A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection,” 2016.
[21] P. W. Rusimamto, Endryansyah, L. Anifah, R. Harimurti, and Y. Anistyasari, “Implementation of arduino pro mini and ESP32 cam for temperature monitoring on automatic thermogun IoT-based,” *Indonesian Journal of Electrical Engineering and Computer Science*, vol. 23, no. 3, pp. 1366–1375, Sep. 2021, doi: 10.11591/ijeecs.v23.i3.pp1366-1375.
[22] E. Syahrudin, E. Utami, and A. D. Hartanto, “Object Detection with YOLOv8 and Enhanced Distance Estimation Using OpenCV for Visually Impaired Accessibility,” *JOIV : International Journal on Informatics Visualization*, vol. 9, no. 2, p. 575, Mar. 2025, doi: 10.62527/joiv.9.2.2826.
[23] B. Smith, “Fast software processing of motion JPEG video,” in *Proceedings of the second ACM international conference on Multimedia - MULTIMEDIA ’94*, New York, New York, USA: ACM Press, 1994, pp. 77–88. doi: 10.1145/192593.192628.
[24] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU Loss: Faster and Better Learning for Bounding Box Regression,” 2016. [Online]. Available: https://github.com/Zzh-tju/DIoU.
[25] A. Navghane, S. Thete, S. Vedpathak, and S. Thag, “WaveGuard: An ESP32-based Ultrasonic Sensing and Edge AI Model for Real-Time Intrusion Detection,” in *3rd International Conference on Emerging Applications of Material Science and Technology, ICEAMST 2025*, Institute of Electrical and Electronics Engineers Inc., 2025, pp. 933–939. doi: 10.1109/ICEAMST67459.2025.11335923.
[26] R. Liu, Y. Hu, H. Zuo, Z. Luo, L. Wang, and G. Gao, “Text-to-Speech for Low-Resource Agglutinative Language With Morphology-Aware Language Model Pre-Training,” *IEEE/ACM Trans. Audio Speech Lang. Process.*, vol. 32, pp. 1075–1087, 2024, doi: 10.1109/TASLP.2023.3348762.
[27] M. M. Sánchez-Sánchez, A. Armenta-Molina, L. H. Hernández-Gómez, and R. T. Rodríguez Márquez, “Implementing Text-to-Speech (TTS)-Based Generated Voice in the Pyttsx3 Synthesizer as Support for People with Speech Impairments,” 2026, pp. 195–204. doi: 10.1007/978-3-032-13551-3_16.
[28] K. Mansoor, M. Afzal, W. Iqbal, and Y. Abbas, “Securing the future: exploring post-quantum cryptography for authentication and user privacy in IoT devices,” *Cluster Comput.*, vol. 28, no. 2, Apr. 2025, doi: 10.1007/s10586-024-04799-4.